

Coding Challenge: Minimize Maximum Pair Sum in Array

Problem Statement

The **pair sum** of a pair (a, b) is equal to $a + b$. The **maximum pair sum** of a list of pairs is the maximum value among all the pair sums.

For example, if we have pairs (1, 5), (2, 3), and (4, 4), the pair sums are $1+5 = 6$, $2+3 = 5$, and $4+4 = 8$. The maximum pair sum of this list would be $\max(6, 5, 8) = 8$.

Given an array `nums` of **even** length n , pair up the elements of `nums` into $n / 2$ pairs such that:

1. Each element of `nums` is in **exactly one** pair.
2. The **maximum pair sum is minimized**.

Return *the minimized maximum pair sum*.

Why Naive Pairing Fails ❌

- **Example List:** `[1, 2, 3, 6]`
- **Bad Strategy:** Pairing adjacent elements or random elements.
 - Pair `(1, 2)` → sum 3
 - Pair `(3, 6)` → sum 9
 - **Max Sum: 9 (Too high!)**

The Optimal Strategy: Greedy 💡

- **Idea:** Pair the **Smallest** with the **Largest**.
- **Why?** This "balances" the sums.
- **Effect:** Prevents two large numbers from summing to a huge value.

1. Advanced & Structural Patterns

- **Test Case 1: Alternating High and Low Values**
 - `nums = [1, 100, 2, 99, 3, 98]`
 - *Expected Output:* 101
 - *Explanation:* Sorting aligns them to `[1, 2, 3, 98, 99, 100]`. Optimal pairs are (1, 100), (2, 99), and (3, 98), all summing perfectly to 101.
- **Test Case 2: Strictly Increasing Arithmetic Progression**
 - `nums = [2, 4, 6, 8, 10, 12]`
 - *Expected Output:* 14
 - *Explanation:* Pairs are (2, 12), (4, 10), and (6, 8). Every single pair sums to exactly 14.

2. Extreme Values & Constraints

- **Test Case 3: All Elements At Maximum Constraint Value (10^5)**
 - `nums = [100000, 100000, 100000, 100000]`
 - *Expected Output:* 200000
 - *Explanation:* Tests for integer overflow thresholds if handled improperly in languages or older types, ensuring the sum correctly reaches 2×10^5 .
- **Test Case 4: Minimal Elements At Lowest Constraint Value (1)**
 - `nums = [1, 1, 1, 1]`
 - *Expected Output:* 2
 - *Explanation:* Validates performance at the absolute floor of the problem constraints.

3. Duplication & Skewed Distributions

- **Test Case 5: Bi-modal Distribution with Massive Duplicate Blocks**
 - `nums = [10, 10, 10, 90, 90, 90]`
 - *Expected Output:* 100
 - *Explanation:* Every 10 is forced to pair with a 90, meaning all pair sums are uniformly 100.
- **Test Case 6: Single Extreme Outlier Pair**
 - `nums = [5, 6, 7, 1, 2, 100]`
 - *Expected Output:* 101
 - *Explanation:* Sorting gives `[1, 2, 5, 6, 7, 100]`. The pairs are (1, 100), (2, 7), and (5, 6). The max sum is heavily dictated by the outlier pair (1, 100) → 101.

Input: `nums = [3, 5, 2, 3]`

Step 1: Sort the array

Sorted: `[2, 3, 3, 5]`

Step 2: Pair extremes (Outer -> Inner)

Pair 1: `(2, 5) -> Sum = 7`

Pair 2: `(3, 3) -> Sum = 6`

Result: `max(7, 6) = 7`